

实验二 Wireshark 抓包实践

一、实验目标

- 学习 Wireshark 的基本使用方法，利用 Wireshark 加深对理论课讲解知识的印象；
- 学习 HTTP、DNS、TCP、UDP 等重要基础协议的数据抓取和过滤，并对结果进行分析；
- 复习 UDP 和 TCP 协议（重点）；
- 鼓励探索计算机网络课外知识，鼓励独立思考和调研。

二、提前阅读

2.1 实验报告书写要求（重要！）

- 1、需要回答问题的部分会使用**红字**标出，例如“**请使用文字与截图（可选）回答**”，没有红色标注的部分不需要回答；
- 2、请将你的回答中的文字设置为“**深蓝色**”，方便批改；
- 3、回答问题时不抄袭、不直接复制大模型的回答、不堆砌文字、保持简洁、体现自己的思考。

2.2 知识补充

- 1、[Wireshark 过滤器使用指南](#)
- 2、[HTTP 的概念、原理、工作机制、数据格式](#)
- 3、[DNS 原理](#)
- 4、[UDP 原理](#)
- 5、[TCP 原理（重点）](#)
- 6、[TCP & UDP 通识视频](#)
- 7、《图解网络-小林 coding》：TCP 篇

注：《图解网络》可在“课程群文件-参考资料”或实验室教师共享盘中下载。

2.3 扩展阅读：如何抓取 HTTPS 数据？

本次实验并不要求大家抓取 HTTPS 数据，此处提供的资料仅供感兴趣的同学参考学习使用：

- 1、[Wireshark 配置 HTTPS 解密](#)（推荐）

2、[Windows 下如何使用 Wireshark 解析 HTTPS 流量](#)

3、[MacOS/Linux 下如何使用 Wireshark 解析 HTTPS 流量](#)

4、其他安全分析工具，如 mitmproxy、Fiddler、Charles、ZAPProxy、Burp Suite 等，都可以用来解析 HTTPS 数据，它们解析 HTTPS 的方法与 Wireshark 不同。这些工具会自己生成一份假的根证书、导入目标设备，从而实现 MITM (man-in-the-middle) 攻击，截获、分析、修改 HTTPS 会话中的数据。这些工具仅可用于开发、测试等合法用途！

三、实验设备

个人主机、Wireshark 软件

注：[Wireshark 下载链接](#)（请根据自己实验平台下载）

四、实验要求

4.1 实验环境配置

- 1) 自行完成 Wireshark 软件的安装；
- 2) 在浏览器中找到并关闭“一律使用安全连接”或类似的设置。该设置会强制使用 HTTPS 连接，与后续需要使用 HTTP 的实验冲突，故需要将其临时关闭。以 Chrome 浏览器为例，该设置位于“设置-隐私与安全-安全-安全连接”下；
- 3) 在进行实验前，关闭有可能产生网络流量的软件，减少干扰；
- 4) 整个实验过程中务必**关闭网络代理（VPN）**！否则代理软件会对数据进行加密，导致很多数据 Wireshark 抓不到，影响实验。

4.2 HTTP 抓包分析

4.2.1 实验步骤

- 1) 打开 Wireshark 并且在捕获选项中选择被抓包的网卡，如下图 1 所示，通常选择 WLAN (Windows) 或 Wi-Fi:en0 (MacOS)。选择完成后 Wireshark 会自动开始抓包。

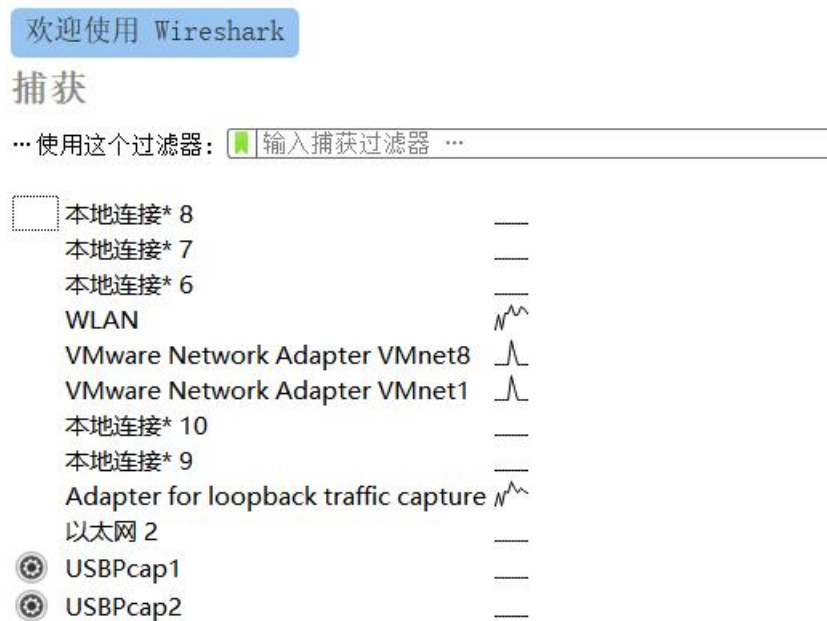


图 1： Wireshark 抓包网卡选择界面

2) 在浏览器输入

<http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>

如果出现如图 2 的界面，说明成功。

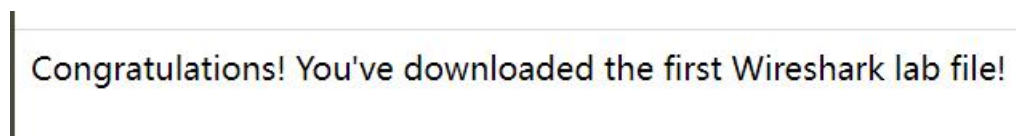


图 2： 服务器返回的 HTML 页面

3) 停止抓包，分析结果。后续实验也均要及时停止抓包，减少不必要的干扰。

4.2.2 问题：请使用文字与截图（可选）回答

1) 结合上节课学过的网络命令，本机和服务器（指 gaia.cs.umass.edu，后面简称为“服务器”）的 ip 地址是什么？

本机 ip: 10.42.239.78

服务器 ip: 128.119.245.12

2) 结合问题 1) 的结果，在搜索栏中筛选出本机跟服务器之间的 http 交互数据（比如输入“ip.addr == 128.119.245.12 and http” [1]，就是筛选出 ip

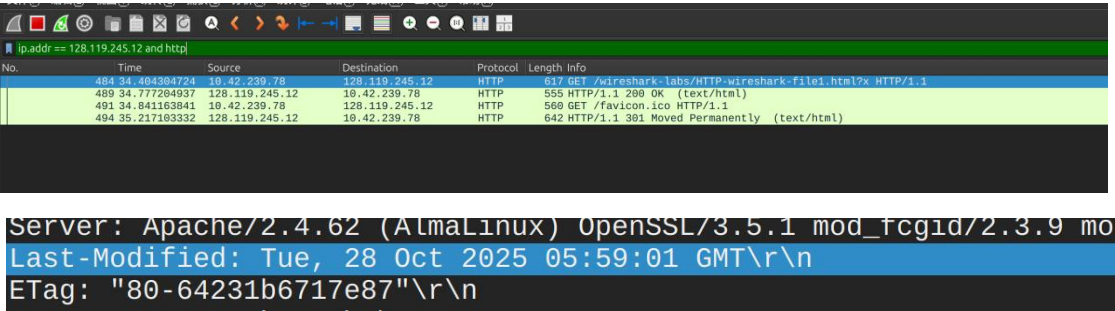
地址为 128.119.245.12 的 http 数据包)，查看过滤结果，找到本机发送请求数据包和服务器返回的响应包，并回答以下问题：

服务器返回的状态码是什么？这个状态码是什么意思？

200 OK;客户端发送的 http 请求已经被服务器相应和处理,并且返回了对应的网页资源.

服务器返回的响应报头中 “Last-Modified” 是什么？

Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT\r\n



请求包和响应包的大小是多少？

617,555

4) 计算从本机发送 http get 请求到 gaia.cs.umass.edu 返回 200 ok 响应的
时间差。提示：可查看 Time 字段，如图 3。

34.777204937 s – 34.404304724 s= 0.372900213 s

No.	Time	Source	Destination	Protocol	Length	Info
2593	13.190336	172.17.101.210	128.119.245.12	HTTP	549	GET /wireshark-labs/INTRO-wireshark-file1.html HTTP/1.1
2662	13.471796	128.119.245.12	172.17.101.210	HTTP	492	HTTP/1.1 200 OK (text/html)

图 3：数据包返回时间查看

关于 Time 字段的含义可通过：视图->时间显示格式来选择或查看，如图 4。

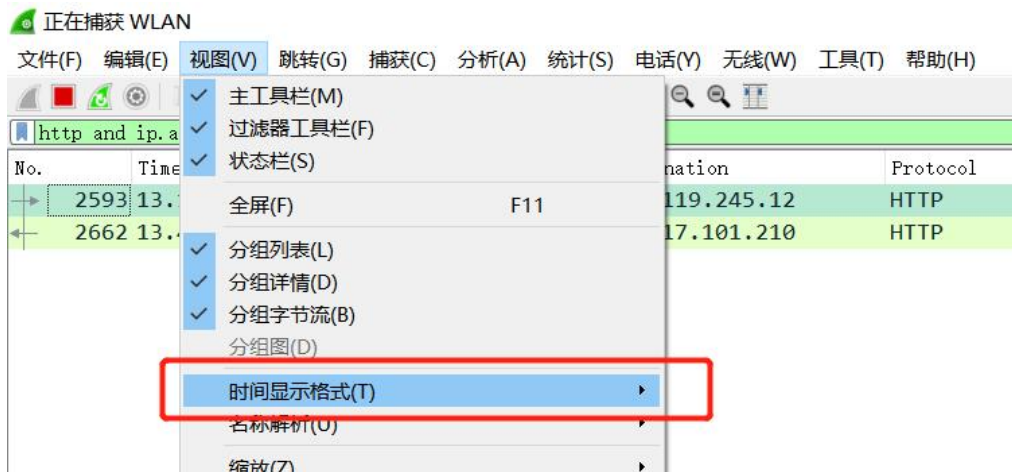


图 4：时间格式选择

4) 再次在浏览器输入实验网址：

<http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>。

观察第一次和第二次本机收到的 **http 响应报文**，是否两次都有“Last-Modified”字段，解释这个字段的作用以及为什么两次呈现不同的情况。
[2]

Last-Modified 字段用于表示资源最后修改时间。浏览器再次请求相同资源时，可能在请求头中携带 If-Modified-Since 字段。如果服务器判断资源自该时间以来没有发生修改，则可能返回 304 Not Modified，从而让浏览器继续使用缓存内容；如果资源已经更新，则服务器会返回 200 OK 和新的资源内容

4.3 DNS 抓包分析 [3]

4.3.1 实验步骤

1) 用 `ipconfig /flushdns` 命令，清空本地 DNS 缓存(该命令较安全可放心使用)。Linux 下如果本身没有安装系统缓存软件的，可忽略这一步。MacOS 系统需使用的命令，请回顾实验 1 的实验要求文档；

2) 打开浏览器，清空浏览器的 DNS 记录。以 Chrome 浏览器为例，从地址栏进入 `chrome://net-internals/#dns`，然后点击“clear host cache”；

3) 打开 Wireshark，在搜索框输入“ip.addr == 自己的 IP 地址”，过滤掉不属于本机的 ip 数据包；

4) 使用 Wireshark, 开始抓包;

5) 利用浏览器访问 “http://www.ietf.org”, 注意观察浏览器地址栏中的协议 (HTTP/HTTPS) 变化;

6) 停止抓包。

4.3.2 问题: 请使用文字与截图 (可选) 回答

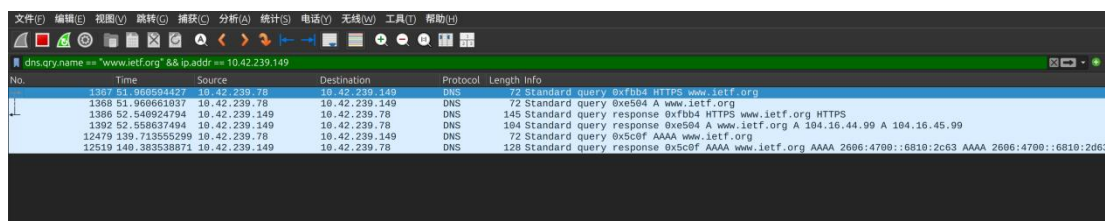
1) 找到查询 “www.ietf.org” 域名的 DNS 查询和响应消息, 它们是通过 UDP 还是 TCP 发送的?

udp

2) DNS 查询消息的目标端口是什么? DNS 响应消息的源端口是什么? 是否一样?

53;53;一样

3) DNS 查询消息发送到哪个 IP 地址? 使用 ipconfig 来确定本地 DNS 服务器的 IP 地址。这两个 IP 地址是否相同?



No.	Time	Source	Destination	Protocol	Length	Info
1367	51.960694427	10.42.239.78	10.42.239.149	DNS	72	Standard query 0x4bb4 www.ietf.org
1368	51.960661037	10.42.239.78	10.42.239.149	DNS	72	Standard query 0xe504 A www.ietf.org
1386	52.540924794	10.42.239.149	10.42.239.78	DNS	145	Standard query response 0x4bb4 HTTPS www.ietf.org
1392	52.558637494	10.42.239.149	10.42.239.78	DNS	104	Standard query response 0xe504 A www.ietf.org A 104.16.44.99 A 104.16.45.99
12479	139.713555299	10.42.239.78	10.42.239.149	DNS	72	Standard query 0x5c9f AAAA www.ietf.org
12519	140.383538871	10.42.239.149	10.42.239.78	DNS	128	Standard query response 0x5c9f AAAA www.ietf.org AAAA 2006:4700::6810:2c63 AAAA 2006:4700::6810:2d63

```
Link 3 (wlp9s0)
  Current Scopes: DNS
    Protocols: +DefaultRoute -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
  Current DNS Server: 10.42.239.149
  DNS Servers: 10.42.239.149
```

10.42.239.149; 当前 dns 服务器地址,10.42.239.149;相同

5) 检查 DNS 查询消息, DNS 查询是什么 "Type"? 查询消息是否包含任何 "answers"?

A 和 AAAA 类型都有.不包含

5) 检查 DNS 响应消息, 它提供了多少个 "answers"? 这些答案具体包含什

```
Frame 24170: Packet, 128 bytes on wire (1024 bits), 128 bytes captured (1024 bits) on interface 0
Ethernet II, Src: 82:a3:80:80:99:c4 (82:a3:80:80:99:c4), Dst: LiteonTechno_68:d9:01 (70:08:99:68:d9:01)
Internet Protocol Version 4, Src: 223.5.5.5, Dst: 10.42.239.78
User Datagram Protocol, Src Port: 53, Dst Port: 57770
  Source Port: 53
  Destination Port: 57770
  Length: 94
  Checksum: 0x0000 [zero-value ignored]
    [Stream index: 278]
    [Stream Packet Number: 2]
  [Timestamps]
  UDP payload (86 bytes)
  Domain Name System (response)
    Transaction ID: 0x3bf8
    Flags: 0x8180 Standard query response, No error
    Questions: 1
    Answer RRs: 2
    Authority RRs: 0
    Additional RRs: 0
    Queries
    Answers
      www.ietf.org: type AAAA, class IN, addr 2606:4700::6810:2c63
      www.ietf.org: type AAAA, class IN, addr 2606:4700::6810:2d63
    [Request In: 24150]
    [Time: 157.732196 milliseconds]
```

么？逐个解释这些答案。

```
Frame 24171: Packet, 104 bytes on wire (832 bits), 104 bytes captured (832 bits) on interface 0
Ethernet II, Src: 82:a3:80:80:99:c4 (82:a3:80:80:99:c4), Dst: LiteonTechno_68:d9:01 (70:08:99:68:d9:01)
Internet Protocol Version 4, Src: 223.5.5.5, Dst: 10.42.239.78
User Datagram Protocol, Src Port: 53, Dst Port: 38108
  Source Port: 53
  Destination Port: 38108
  Length: 70
  Checksum: 0x0000 [zero-value ignored]
    [Stream index: 279]
    [Stream Packet Number: 2]
  [Timestamps]
  UDP payload (62 bytes)
  Domain Name System (response)
    Transaction ID: 0xf157
    Flags: 0x8180 Standard query response, No error
    Questions: 1
    Answer RRs: 2
    Authority RRs: 0
    Additional RRs: 0
    Queries
    Answers
      www.ietf.org: type A, class IN, addr 104.16.45.99
      www.ietf.org: type A, class IN, addr 104.16.44.99
    [Request In: 24151]
    [Time: 157.734912 milliseconds]
```

对于发送给 223.5.5.5 的 dns 请求,A 和 AAAA 类查询分别返回了两个 ipv4(104.16.45.99,104.16.44.99)和两个 ipv6 地址(2606:4700::6810:2c63 和 2606:4700::6810:2d63),对于发送给 119.29.29.29 也是类似。

6) 考虑从主机发送的后续 TCP SYN 数据包。这些 SYN 数据包的目的 IP 地址

No.	Time	Source	Destination	Protocol	Length	Info
24174	439.989115279	10.42.239.78	104.16.44.99	TCP	74	32928 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2983655077 TSecr=0 WS=128
24238	431.360107237	10.42.239.78	104.16.44.99	TCP	74	32945 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2983655448 TSecr=0 WS=128
24475	432.183281420	10.42.239.78	104.16.44.99	TCP	74	32952 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2983656271 TSecr=0 WS=128
24173	439.888483948	10.42.239.78	104.16.45.99	TCP	74	35970 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=2185131547 TSecr=0 WS=128

是否与 DNS 响应消息中提供的任何 IP 地址相对应？

是的,两个 ip 地址都相同

7) 在进行实验步骤 5) 时, 你观察到了什么变化? 上网查找相关资料, 解释为什么会产生这种现象? 提示: 观察你输入的和浏览器最终使用的协议, 是 HTTP 还是 HTTPS?

http 连接变成了 https 连接;

网站从 HTTP 转为 HTTPS 通常有两种实现方式:一种是服务器收到 HTTP 请求后, 返回重定向响应, 引导浏览器访问 HTTPS 地址;另一种是浏览器已经保存或预置了该站点的 HSTS 策略, 直接将 HTTP 请求升级为 HTTPS。HSTS 是 RFC 6797 定义的机制, 允许网站声明浏览器后续只能通过安全连接访问该站点。HSTS 的机制由 RFC 6797 定义: 站点可以要求用户代理仅通过安全连接与其通信, 并通过 Strict-Transport-Security 响应头声明相关策略。

其他提示:

- 1、可以通过 `ipconfig /all` 查看本机网口的 IP 地址、MAC 地址、DNS 服务器等信息
- 2、可以通过 `nslookup` 命令查看目的域名对应的 ip 地址, 如图 5 查看 `baidu.com` 对应的 ip 地址。

```
PS C:\Users\user> nslookup baidu.com
服务器:  UnKnown
Address:  10.8.4.4

非权威应答:
名称:     baidu.com
Addresses: 220.181.38.251
           220.181.38.148
```

图 5: nslookup 查看域名对应的 ip 地址

3、如果你的设备、网络环境均支持 IPv6, 则第一次 DNS 请求查询域名的 Type 对应 IPv4 地址 (A 代表 IPv4), 第二次 DNS 请求查询域名对应 IPv6 地址 (AAAA 代表 IPv6)

4.4 UDP 抓包分析

4.4.1 实验步骤

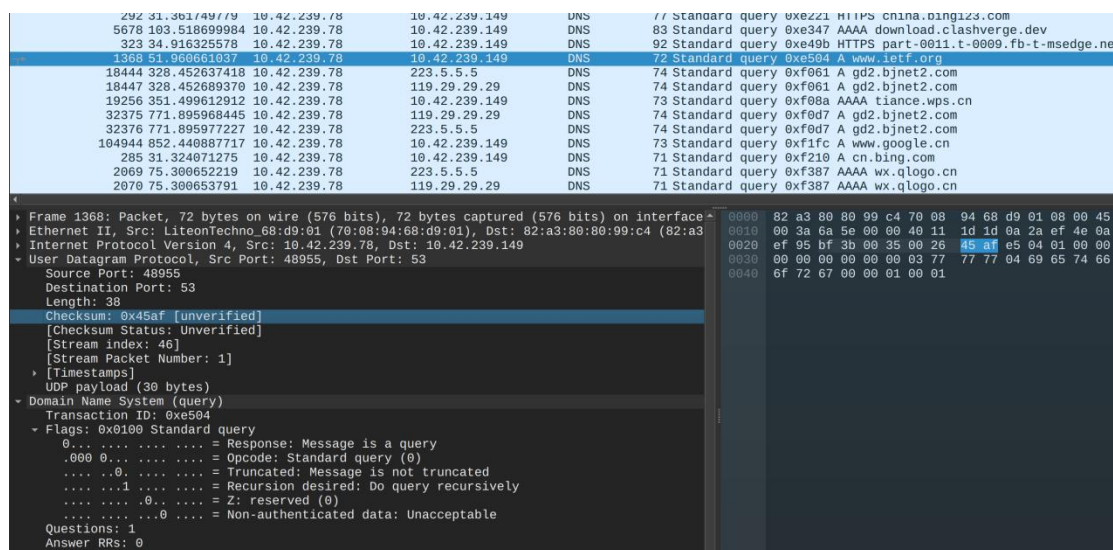
1) 打开 Wireshark 开始抓包, 然后执行会导致主机发送和接收 UDP 数据包的操作 (比如提前动手写一个 UDP 发包程序并运行、执行 DNS 查询、用浏览器访问网页等, 任选其一即可)

2) 停止抓包, 把 UDP 数据包过滤出来。

4.4.2 问题: 请使用文字与截图 (可选) 回答

1) 任意选择一个 UDP 数据包, 观察 UDP 报头中有哪些字段, 逐个解释这些字段;

Source Port: 源端口号, 客户端向服务器端发送报文所用的临时端口号。



Destination Port: 目的端口号, 目标服务器对应的端口号

Length: udp 数据包总长度 (首部+负载)

CheckSum: udp 校验和, 检测传输过程是否发生错误。

2) 观察 UDP 报头中各个字段的信息, 请问 UDP 报头长度是多少?

2+2+2+2=8(byte)

3) UDP 报头中的长度字段 (Length) 是什么意思, 请结合你抓到的 UDP 数据包回答。

length(38)是整个 udp 数据报的长度,包括首部(8)和负载长度(30);

4) 结合长度字段的比特长度大小, 思考 UDP 报文的最大负载是多少?

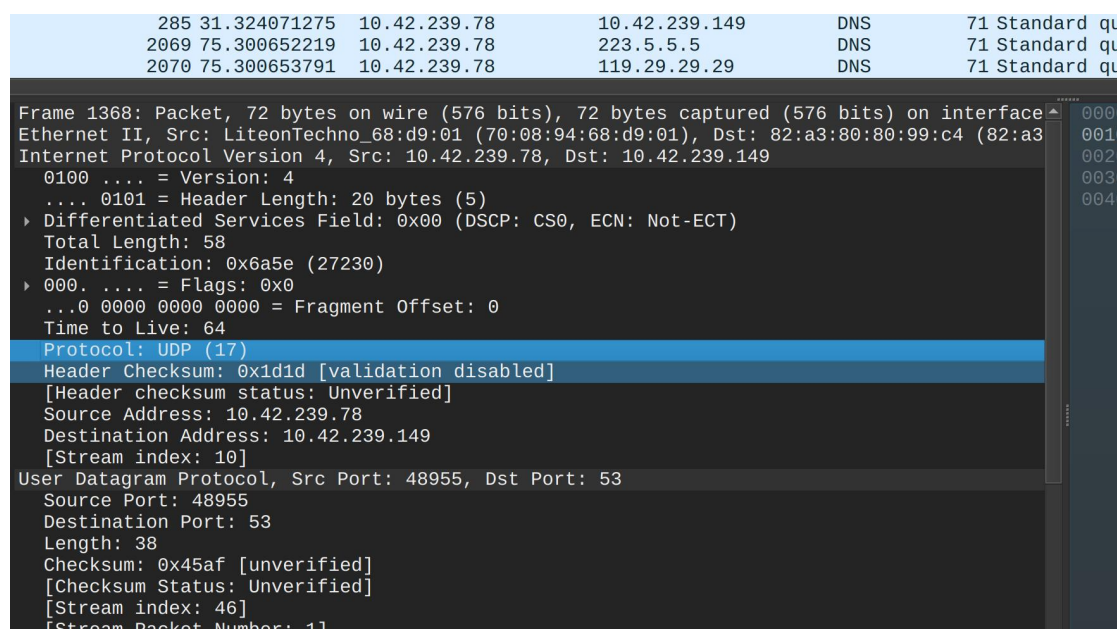
65535 - 20 (ipv4 首部) - 8(udp 首部) = 65507 字节

5) 你选择的 UDP 数据报源端口和目的端口是什么? 思考端口号范围大小是什么?

48955,53; 0-65535

6) UDP 的协议号是什么? 以十六进制和十进制表示法给出答案。提示: 在 IP 层找 Protocol 字段。

17;0x11



7) 观察一对 UDP 请求与响应包, 描述两者间端口号的联系。

请求包的源端口 = 响应包的目的地端口

请求包的目的地端口 = 响应包的源端口

4.5 TCP 抓包分析: 三次握手

4.5.1 实验步骤

1) 清理浏览器的缓存。以谷歌浏览器为例: 历史记录-历史记录-清除浏览数据-清除图片和文件

2) 打开 Wireshark 并启动抓包, 在浏览器中输入:

<http://gaia.cs.umass.edu/wireshark-labs/wireshark-traces.zip>

待文件下载完后，停止抓包。由于使用了不安全的连接（HTTP），一些浏览器可能阻止本次下载，点击下载界面中的“保留”按钮即可。

3) 把本机与“gaia.cs.umass.edu”（下面简称“服务器”）之间发送的所有数据包过滤出来（过滤条件：ip.addr==服务器 IP 地址，确定服务器 IP 地址的方式前面已经介绍过）

4.5.2 问题：请使用文字与截图（可选）回答

1) 本机是利用哪个 IP 和端口与服务器进行交互的？

No.	Time	Source	Destination	Protocol	Length	Info
250	5.293425285	10.42.239.78	128.119.245.12	TCP	74	50238 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=290286598 TSecr=0 WS=128
256	5.568399028	128.119.245.12	10.42.239.78	TCP	74	80 → 50238 [SYN, ACK] Seq=0 Ack=1 Win=65160 Len=0 MSS=1460 SACK_PERM TSval=1229992463 TSecr=290286598
257	5.568450224	10.42.239.78	128.119.245.12	TCP	66	50238 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=290286153 TSecr=1229992463
268	5.568772742	10.42.239.78	128.119.245.12	HTTP	499	GET /wireshark-labs/wireshark-traces.zip HTTP/1.1
290	6.003629242	128.119.245.12	10.42.239.78	TCP	66	80 → 50238 [ACK] Seq=1 Ack=434 Win=64768 Len=0 TSval=1229992901 TSecr=290286153
292	6.005494409	128.119.245.12	10.42.239.78	TCP	2962	80 → 50238 [PSH, ACK] Seq=1 Ack=434 Win=64768 Len=2896 TSval=1229992901 TSecr=290286153 [TCP PD
293	6.005494510	128.119.245.12	10.42.239.78	TCP	1514	80 → 50238 [ACK] Seq=2897 Ack=434 Win=64768 Len=1448 TSval=1229992901 TSecr=290286153 [TCP PDU
294	6.005509446	10.42.239.78	128.119.245.12	TCP	66	50238 → 80 [ACK] Seq=434 Ack=2897 Win=70144 Len=0 TSval=290286590 TSecr=1229992901
295	6.005510519	10.42.239.78	128.119.245.12	TCP	66	50238 → 80 [ACK] Seq=434 Ack=4345 Win=72960 Len=0 TSval=290286590 TSecr=1229992901
296	6.005559570	128.119.245.12	10.42.239.78	TCP	1514	80 → 50238 [PSH, ACK] Seq=4345 Ack=434 Win=64768 Len=1448 TSval=1229992901 TSecr=290286153 [TCP
297	6.005600512	10.42.239.78	128.119.245.12	TCP	66	50238 → 80 [ACK] Seq=434 Ack=5793 Win=75904 Len=0 TSval=290286590 TSecr=1229992901
298	6.007819379	128.119.245.12	10.42.239.78	TCP	2962	80 → 50238 [PSH, ACK] Seq=5793 Ack=434 Win=64768 Len=2896 TSval=1229992901 TSecr=290286153 [TCP
299	6.007819623	128.119.245.12	10.42.239.78	TCP	1514	80 → 50238 [ACK] Seq=8689 Ack=434 Win=64768 Len=1448 TSval=1229992901 TSecr=290286153 [TCP PDU
300	6.007843364	10.42.239.78	128.119.245.12	TCP	66	50238 → 80 [ACK] Seq=434 Ack=8689 Win=81664 Len=0 TSval=290286593 TSecr=1229992901
301	6.007852816	10.42.239.78	128.119.245.12	TCP	66	50238 → 80 [ACK] Seq=434 Ack=10137 Win=84608 Len=0 TSval=290286593 TSecr=1229992901
302	6.008030340	128.119.245.12	10.42.239.78	TCP	1514	80 → 50238 [PSH, ACK] Seq=10137 Ack=434 Win=64768 Len=1448 TSval=1229992901 TSecr=290286153 [TC
303	6.008046847	10.42.239.78	128.119.245.12	TCP	66	50238 → 80 [ACK] Seq=434 Ack=11585 Win=87424 Len=0 TSval=290286593 TSecr=1229992901
304	6.009237051	128.119.245.12	10.42.239.78	TCP	2962	80 → 50238 [PSH, ACK] Seq=11585 Ack=434 Win=64768 Len=2896 TSval=1229992902 TSecr=290286153 [TC
305	6.009244056	10.42.239.78	128.119.245.12	TCP	66	50238 → 80 [ACK] Seq=434 Ack=14481 Win=86784 Len=0 TSval=290286594 TSecr=1229992902
349	6.274958592	128.119.245.12	10.42.239.78	TCP	2962	80 → 50238 [PSH, ACK] Seq=14481 Ack=434 Win=64768 Len=2896 TSval=1229993168 TSecr=290286590 [TC
350	6.274958978	128.119.245.12	10.42.239.78	TCP	2962	80 → 50238 [PSH, ACK] Seq=17377 Ack=434 Win=64768 Len=2896 TSval=1229993168 TSecr=290286590 [TC
351	6.274959000	128.119.245.12	10.42.239.78	TCP	2962	80 → 50238 [PSH, ACK] Seq=20728 Ack=434 Win=64768 Len=2896 TSval=1229993168 TSecr=290286590 [TC
352	6.274959084	128.119.245.12	10.42.239.78	TCP	2962	80 → 50238 [PSH, ACK] Seq=23109 Ack=434 Win=64768 Len=2896 TSval=1229993168 TSecr=290286590 [TC

10.42.239.78:50234;

2) 服务器是利用哪个 IP 和端口与本机进行交互的？

128.119.245.12: 80;

Frame 250: Packet, 74 bytes on wire (592 bits), 74 bytes captured (592 bits) on Ethernet II, Src: LiteonTechno_68:d9:01 (70:08:94:68:d9:01), Dst: 6a:4f:e2:da:9
Internet Protocol Version 4, Src: 10.42.239.78, Dst: 128.119.245.12
Transmission Control Protocol, Src Port: 50238, Dst Port: 80, Seq: 0, Len: 0
Source Port: 50238
Destination Port: 80
[Stream index: 41]
[Stream Packet Number: 1]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 2410528135
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1010 = Header Length: 40 bytes (10)
Flags: 0x002 (SYN)
Window: 64240
[Calculated window size: 64240]
Checksum: 0x51cb [unverified]
Checksum Status: Unverified
Urgent Pointer: 0
Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Op
[Timestamps]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]

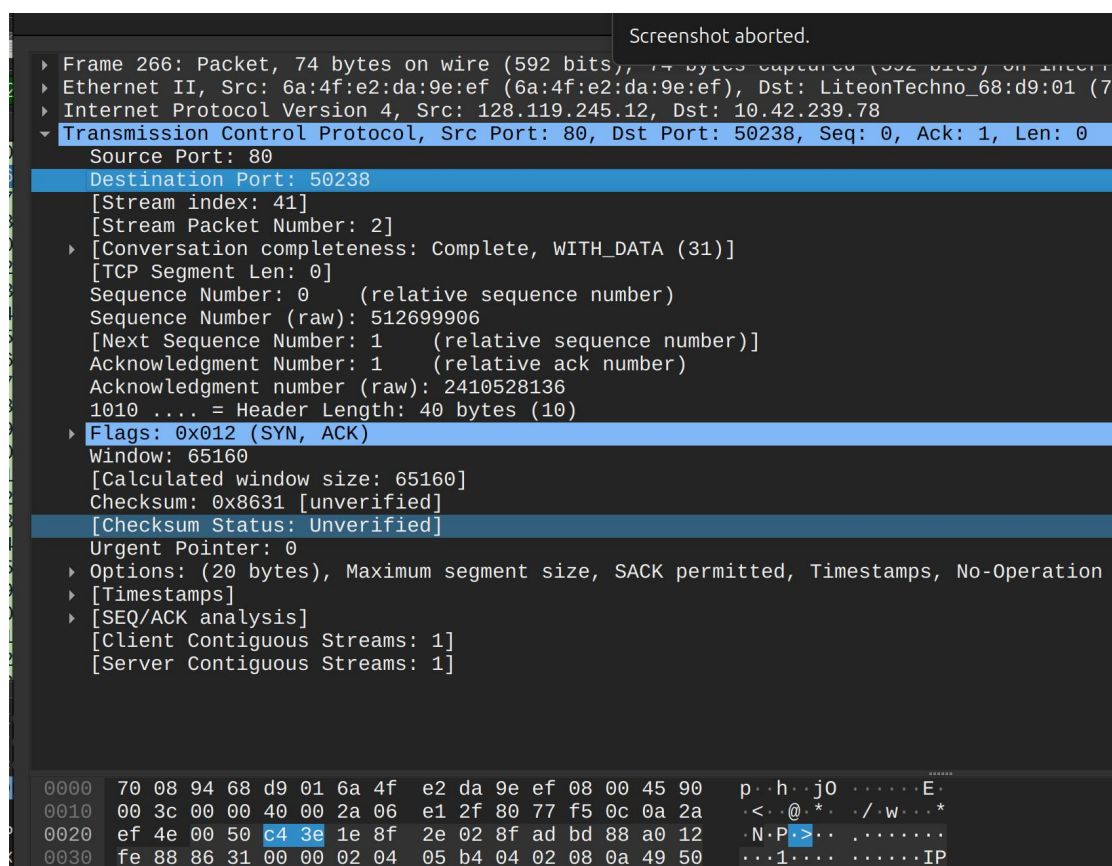
3) 找到本机与服务器第一次握手的数据包，你是如何确定这是第一次握手的数据包的？

250 帧是 tcp 三次握手的第一次握手，报文中含有 SYN 标志

4) 分析 3) 中你找到的第一次握手数据包，其序列号 (seq) 是多少？注意：Wireshark 对于初始序列号会有两个描述，一个是 relative，一个是 raw，我们这里指 raw，原始数据。

2410528135

5) 找到服务器与本机第二次握手的数据包，你是如何确定这是第二次握手



的数据包的？

266 帧报文中含有[SYN,ACK]两个标志

6) 分析 5) 中你找到的第二次握手数据包，请问其序列号 (seq) 是多少？确认号字段 (ack) 的值是多少？ack 的值是由什么决定的？

512699906；2410528136；第二次握手中的确认号由第一次握手中客户端发送的初始序列号决定；

7) 找到服务器与本机第三次握手的数据包，你是如何确定这是第三次握手的数据包的？

报文在 syn 和[syn,ack]标志之后，并且是从 50234 发往 80 端口，含有 ack 标志

```
Wireshark - 分组 267 - wlp9s0
Frame 267: Packet, 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface wlp9s0, id 0
  Ethernet II, Src: LiteonTechno_68:d9:01 (70:08:94:68:d9:01), Dst: 6a:4f:e2:da:9e:ef (6a:4f:e2:da:9e:ef)
  Internet Protocol Version 4, Src: 10.42.239.78, Dst: 128.119.245.12
  Transmission Control Protocol, Src Port: 50238, Dst Port: 80, Seq: 1, Ack: 1, Len: 0
    Source Port: 50238
    Destination Port: 80
    [Stream index: 41]
    [Stream Packet Number: 3]
    [Conversation completeness: Complete, WITH_DATA (31)]
    [TCP Segment Len: 0]
    Sequence Number: 1 (relative sequence number)
    Sequence Number (raw): 2410528136
    [Next Sequence Number: 1 (relative sequence number)]
    Acknowledgment Number: 1 (relative ack number)
    Acknowledgment number (raw): 512699907
    1000 ..... = Header Length: 32 bytes (8)
    Flags: 0x010 (ACK)
    Window: 502
    [Calculated window size: 64256]
    [Window size scaling factor: 128]
    Checksum: 0xb07d [unverified]
    [Checksum Status: Unverified]
    Urgent Pointer: 0
    Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
    [Timestamps]
    [SEQ/ACK analysis]
    [Client Contiguous Streams: 1]
    [Server Contiguous Streams: 1]
    0000 6a 4f e2 da 9e ef 70 08 94 68 d9 01 08 00 45 00 j0...p...h...E
    0010 00 24 70 f5 40 00 40 06 4c d2 03 20 0f 40 80 77 4s...@...L...N...
```

8) 分析 7) 中你找到的第三次握手数据包，请问其序列号 (seq) 是多少？结合第一次和第二次握手的序列号、确认号以及包长度，解释为什么第三次握手的序列号和确认号是这个值？

2410528136; 512699907;

第三次握手序列号的来源：第一次握手中，客户端发送的原始序列号为：2410528135，由于第一次握手发送了 SYN 报文，而 SYN 会占用一个序列号，因此第三次握手中客户端继续发送报文时，其原始序列号应为：2410528135 + 1 = 2410528136 ；

第三次握手确认号的来源 第二次握手中，服务器发送的原始序列号为：512699906； 由于第二次握手同样发送了 SYN 报文，服务器的 SYN 也会占用一个序列号，因此第三次握手中客户端对服务器的确认号应为：512699906 + 1 = 512699907

9) 观察三次握手之后的普通 TCP 数据报文，其与三次握手报文的最大区别是什么？提示：从报头的字段来进行解释。

三次握手报文用于建立 TCP 连接，主要通过 SYN 和 ACK 标志位完成双方的连接确认，通常不携带实际应用层数据，因此其 TCP 数据长度为 0；三次握手之后的普通 TCP 数据报文用于传输 HTTP 请求和服务器返回的文件内容，其数据长度通常大于 0，并可能设置 PSH、ACK 等标志位。

五、课后练习题

请使用文字与截图（可选）回答以下问题。

1) 在使用 Wireshark 抓包时，你可能观察到一些先前未详细学习过的协议（如 QUIC）。请自行查阅相关资料，在此简要介绍一个或两个课本中没有详细学习的协议。例如：它们属于哪一层、主要用于什么场景、有什么优缺点。

QUIC 是一种基于 UDP 的安全传输协议，从功能上看属于传输层协议。它能够为应用程序提供可靠传输、流量控制、多路复用、安全通信以及连接迁移等功能。HTTP/3 正是将 HTTP 通信建立在 QUIC 协议之上。

QUIC 主要应用于现代网页访问，特别是 HTTP/3 场景。与传统的 HTTP/2 + TCP + TLS 通信方式相比，QUIC 基于 UDP 实现传输控制，并将安全连接建立机制纳入协议设计中，从而有助于降低连接建立时延。

QUIC 也存在一定缺点。由于其大部分控制信息和应用数据受到加密保护，中间网络设备难以像观察 TCP 那样直接分析其传输状态，这会增加网络监控和故障排查的难度。另外，QUIC 基于 UDP，在某些限制 UDP 流量的网络环境中可能无法正常发挥作用，客户端可能需要回退到基于 TCP 的通信方式。

2) 简述从浏览器输入域名开始，计算机是如何解析出目的 IP 地址的？

首先，浏览器会检查自身是否已经缓存了该域名对应的 IP 地址。如果浏览器缓存中没有可用结果，操作系统还可能检查本地 DNS 缓存以及本地静态映射文件，例如 Linux 系统中的 /etc/hosts 文件。

如果本地仍然无法得到解析结果，主机会向当前配置的 DNS 服务器发送 DNS 查询报文。DNS 服务器收到查询请求后，会先检查自身缓存。如果缓存中已经存在有效结果，就直接将解析结果返回给本机；如果没有缓存结果，则由 DNS 解析系统逐步查询相关域名服务器，最终获取该域名对应的地址记录，并将结果返回给客户端。

3) 查阅相关资料，简要解释响应头中一些有关缓存的字段的作用，如：Cache-Control、Expires、Last-Modified、ETag。浏览器和服务器如何使用这些字段完成资源缓存功能的？

Cache-Control 是控制缓存策略的重要字段。例如，服务器返回：Cache-Control: max-age=3600 表示浏览器在接下来的 3600 秒内可以将该资源视为仍然有效，通常不需要重新向服务器获取完整内容。若服务器返回：Cache-Control: no-cache

并不是表示完全不能保存资源,而是表示该资源在再次使用前必须先与服务器进行验证。若返回: Cache-Control: no-store 则表示浏览器不应保存该响应内容。

Expires 字段同样用于描述缓存有效期,但它使用的是一个具体的绝对时间,例如: Expires: Wed, 27 May 2026 12:00:00 GMT 在到达该时间之前,浏览器可以认为缓存资源仍未过期。与 Expires 相比, Cache-Control: max-age 采用相对秒数表示有效期,控制方式更灵活;

当响应中同时存在两者时, Cache-Control 中的有效期规则优先于 Expires。

Last-Modified 表示资源最后一次被修改的时间。例如,服务器第一次返回网页资源时可能在响应头中附带: Last-Modified: Tue, 26 May 2026 08:00:00 GMT 浏览器保存资源及该时间后,下一次访问相同资源时,可以在请求头中加入: If-Modified-Since: Tue, 26 May 2026 08:00:00 GMT 如果服务器判断资源自该时间后没有改变,可以返回: 304 Not Modified 此时浏览器继续使用本地缓存,不需要重新接收完整资源;如果资源已经改变,服务器则返回新的资源内容。

ETag 是服务器为资源当前版本生成的标识符。例如,服务器第一次返回资源时可能携带: ETag: "abc123" 浏览器再次访问时,可以发送: If-None-Match: "abc123" 服务器比较当前资源的 ETag 与浏览器发送的值: 如果一致,说明资源版本没有变化,服务器可以返回 304 Not Modified; 如果不一致,则返回新的资源内容和新的 ETag。与按修改时间判断的 Last-Modified 相比, ETag 可以更准确地区分资源内容版本。

当请求中同时存在 If-None-Match 和 If-Modified-Since 时,服务器应优先依据 If-None-Match 进行判断。

4) 查阅相关资料,简要解释强缓存、协商缓存的区别;

强缓存是指浏览器在本地缓存资源仍处于有效期内时,直接使用本地资源,不再向服务器发送请求。服务器通常通过响应头中的 Cache-Control 或 Expires 来规定缓存的有效时间。例如, Cache-Control: max-age=3600 表示浏览器在接下来的 3600 秒内可以直接使用缓存资源。此时不需要与服务器通信,因此访问速度更快,也最节省网络流量。

协商缓存是指浏览器已经保存了资源,但在再次使用前,需要先向服务器确认本地缓存是否仍然有效。浏览器会在请求中携带缓存验证信息,例如使用 If-Modified-Since 对应服务器此前返回的 Last-Modified,或使用 If-None-Match 对应服务器此前返回的 ETag。服务器验证后,如果资源没有变化,就返回 304 Not Modified,浏览器继续使用本地缓存;如果资源已经变化,则返回 200 OK 和新的资源内容。

5) 观察多次建立 TCP 连接的初始序列号,我们发现 TCP 报文的初始序列号

都是一串随机数 (relative seq 本身并不在 TCP 报文中, 它是 Wireshark 软件额外算出来的, 目的是方便我们观察), 请问 TCP 为什么这么设计?

TCP 通过序列号对传输的数据字节进行编号, 以便接收方按正确顺序重组数据, 并完成确认、重传和重复报文丢弃等功能。在每次建立 TCP 连接时, 通信双方都要分别选择一个初始序列号, 即 ISN (Initial Sequence Number)。

TCP 的初始序列号不会固定从 0 开始, 而是选择为难以预测、看起来随机的数值, 主要有以下两个原因。

第一, 防止旧连接中的延迟报文干扰新连接。网络中可能存在因延迟、重传或连接异常而残留的旧 TCP 报文。如果新的 TCP 连接仍然从固定序列号开始, 且源 IP、源端口、目的 IP、目的端口等连接信息再次相同, 那么旧连接中的延迟报文可能被误认为是新连接中的有效数据。通过让每次连接使用不同的初始序列号, 可以降低旧报文与新连接有效序列号范围发生混淆的可能性, 从而保证数据传输的正确性。

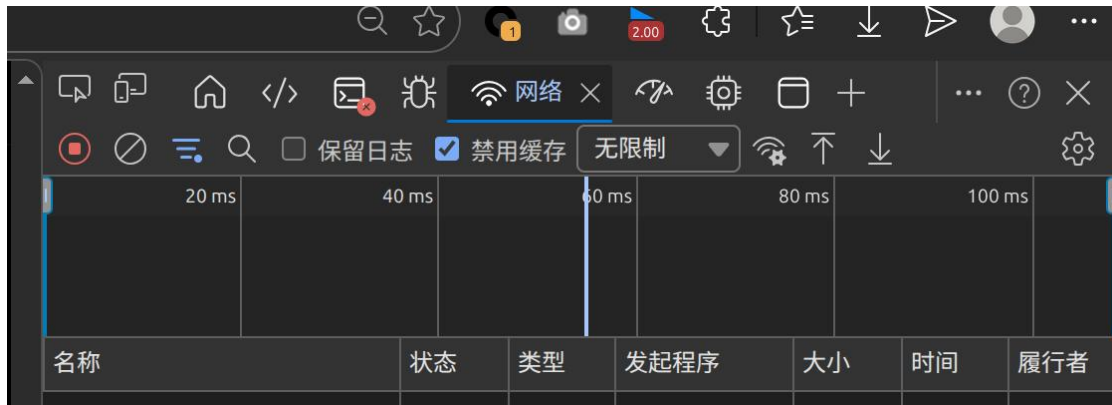
第二, 提高连接的安全性。如果 TCP 初始序列号固定或容易预测, 攻击者可能猜出某条连接正在使用的序列号, 从而伪造 TCP 报文, 尝试注入数据、干扰连接或冒充通信一方。采用难以预测的初始序列号, 可以明显增加攻击者在不知道真实通信内容的情况下伪造有效 TCP 报文的难度。

6) 为什么 TCP 握手必须要三次, 两次、四次可以吗? 从其优缺点来分析。

两次握手的问题是: 服务器无法确认客户端已经收到服务器的初始序列号; 不能有效避免旧的、延迟到达的连接请求造成错误连接; 可能导致服务器建立无效连接并浪费资源。

四次握手在逻辑上可以建立连接, 但其中服务器发送的确认报文和同步报文可以合并为一次 SYN, ACK, 因此四次握手会产生不必要的额外开销。

7) 探索你所使用的浏览器, 简要介绍: 如何打开开发者工具? 如何关闭浏览器的缓存功能? 如何在浏览器中管理 cookie? 可选: 浏览器中你还发现了什么有趣的功能, 简单介绍一下?



右键检查，f12,右上角浏览器-》开发者选项（我自己不问 ai 也都知道）

管理 cookie：

1. 设置-》隐私和安全-》第三方 cookie
2. f12-》应用程序面板-》存储-》cookie；

有趣的功能：

之前某知名网站有过 mimo 的额度无限获取的 bug（到现在还能用），只需要从小米官方登陆账号，然后获取 userid 和 username 等信息，然后填入某大佬开发的连接，就能无限使用 mimo 了。我觉得手动填写太麻烦，了解到开发者选项里有控制台，写了个获取几个信息的脚本。（单纯提取一些文字并组织一下而已）

其次，还有个长连接的使用，像 gpt 这些订购有很严格的检测，使用长连接就可以跳过一些比较严格的检测。

六、参考

- [1] Wiki.wireshark.org. 2022. DisplayFilters. [online] Available at: <<https://wiki.wireshark.org/DisplayFilters>> [Accessed 26 February 2022].
- [2] Segmentfault.com. 2022. 一张图理解 Http 缓存 - SegmentFault 思否. [online] Available at: <<https://segmentfault.com/a/1190000015816331>>
- [3] Cnblogs.com. 2022. wireshark dns 域名解析抓包实验分析 - 不忘初心 dbstdxq - 博客园. [online] Available at: <<https://www.cnblogs.com/mggahui/p/13899888.html>> [Accessed 26

February fds022].

七、提交要求

1、实验报告：完成所有需要回答的部分（已使用**红字**指出），适当配合截图加以说明。你的回答应尽可能条理清晰，截图请缩放至适当大小；

2、提交截止时间：2026/5/27 中午 11:59:59

3、提交方式：将该文档**转换为 PDF**，上传至坚果云。注意**正确填写坚果云界面中要求的姓名、学号等字段**，系统会自动重命名。只需提交 PDF 实验报告。

4、提交链接：[实验课小实验报告统一收集](#)